

Single-round Lattice-based Multisignatures

Kittiphon Phalakarn
Dept. of Electrical and Computer Eng.
University of Waterloo
kphalakarn@uwaterloo.ca

Vorapong Suppakitpaisarn
Dept. of Computer Science
The University of Tokyo
vorapong@is.s.u-tokyo.ac.jp

M. Anwar Hasan
Dept. of Electrical and Computer Eng.
University of Waterloo
ahasan@uwaterloo.ca

Abstract—This work presents a provably-secure lattice-based multisignature scheme which requires only a single round of communication, whereas the existing works need two or three rounds. The reduction in the number of rounds for the proposed scheme is achieved by utilizing lattice trapdoors. In order to generate multisignatures securely, our scheme however requires an honest centralized server that maintains the trapdoor of a shared matrix used in the scheme.

Index Terms—multisignatures, trapdoors, lattice-based cryptography, post-quantum cryptography

I. INTRODUCTION

Multisignatures [1] are proofs that a group of N users have agreed on and signed the same message. We can have each user sign the message independently, but that results in N signatures and we have to perform the verification N times. Using multisignature schemes, we require only one signature for the whole group and one execution of the verification algorithm. Multisignatures are very useful in blockchains and cryptocurrencies. For example, by having one signature for the whole group in a transaction, the amount of data written on the blockchain and sent in the network is reduced.

Currently, there exist a variety of multisignature schemes based on the hardness of integer factorization (i.e., RSA) [2] and discrete logarithm [3]. As the cryptographic schemes based on any of these hard problems can be broken by large quantum computers, many researchers are turning to post-quantum cryptography. Lattice-based cryptosystem [4] is considered quantum-safe and has received much interest in the research community due to its moderate-sized ciphertext/signatures and simple computations. Nevertheless, lattice-based multisignatures are still not practical for real-world use and have rooms for improvements.

The first lattice-based multisignature scheme was proposed by Bansarkhani and Sturm [5]. It is a three-round protocol that requires signers to broadcast their information. The protocol was improved by subsequent works [6], [7] and the recent scheme presented by Damgård et al. [8] has reduced the number of rounds to two. All of the aforementioned schemes are derived from the lattice-based signatures by Lyubashevsky et al. [9]–[11] based on the Fiat-Shamir transform.

Here, we point out that broadcasting limits the scalability of the schemes. It is specially expensive in blockchain applications with up to 100 signers [12]. To the best of our knowledge, no secure single-round lattice-based multisignature scheme without broadcasting has been proposed. There was an attempt to do so [13], but it was later proved to be insecure [14].

A. Contributions

In this work, we propose a *single-round non-broadcasting* lattice-based multisignature scheme based on the Fiat-Shamir transform. In order to restrict the number of rounds to one, we utilize lattice trapdoors by Micciancio and Peikert [15]. For easier understanding, we start by presenting a signature scheme and then extend it to a multisignature one. Security proofs based on the general forking lemma are included for both of our schemes.

Our proposed multisignature scheme has two advantages: (i) no broadcasting is required, and (ii) if any signer has to restart the protocol, other signers do not have to restart their parts, unlike the existing schemes where all signers are affected. However, an honest centralized server maintaining the trapdoor of a shared matrix is required in order to work securely.

II. PRELIMINARIES

A. Signature Scheme

A signature scheme Sig consists of three algorithms:

- $(pk, sk) \xleftarrow{\$} \text{Sig.KeyGen}(1^\lambda)$: The key generation algorithm produces a key pair from a security parameter λ .
- $\sigma \xleftarrow{\$} \text{Sig.Sign}(sk, \mu)$: The signing algorithm uses a secret key sk to create a signature σ for a message μ .
- $b \leftarrow \text{Sig.Verify}(pk, \mu, \sigma)$: The verification algorithm outputs $b = 1$ if a signature σ is valid for a public key pk and a message μ . Otherwise, it outputs $b = 0$.

A signature scheme is *correct* if all signatures created by the signing algorithm pass the verification, i.e., for all $\lambda \in \mathbb{N}$ and $\mu \in \{0, 1\}^*$, it holds, over all possible (pk, sk) generated from $\text{Sig.KeyGen}(1^\lambda)$, that $\Pr[\text{Sig.Verify}(pk, \mu, \text{Sig.Sign}(sk, \mu)) = 1] = 1$.

A signature scheme is *existentially unforgeable under one-per-message chosen-message attacks* [16] if for any probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$, the advantage $\text{Adv}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda) = \Pr[\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda)]$ is $\text{negl}(\lambda)$.

B. Multisignature Scheme

In this setting, there are N signers agreeing to sign a message μ . A multisignature scheme MS consists of five algorithms:

- $(pk_i, sk_i) \xleftarrow{\$} \text{MS.KeyGen}(1^\lambda)$: The key generation algorithm produces a key pair for each signer.
- $\sigma_i \xleftarrow{\$} \text{MS.Sign}(sk_i, PK, \mu)$: The signing algorithm creates a signature σ_i of the i -th signer where $PK = \{pk_j\}_{j=1}^N$.

Experiment $\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda)$

```

1:  $\mathcal{M} \leftarrow \{\}$ 
2:  $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{Sig.KeyGen}(1^\lambda)$ 
3:  $(\mu^*, \sigma^*) \xleftarrow{\$} \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(1^\lambda, \text{pk})$ 
4: if  $\text{Sig.Verify}(\text{pk}, \mu^*, \sigma^*) = 1$  and  $\mu^* \notin \mathcal{M}$  then
5:   return 1
6: else return 0

```

$\text{Sign}(\text{sk}, \mu)$

```

1: if  $\mu \in \mathcal{M}$  then return  $\perp$ 
2:  $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mu\}$ 
3: return  $\text{Sig.Sign}(\text{sk}, \mu)$ 

```

Experiment $\text{Exp}_{\text{MS}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda)$

```

1:  $\mathcal{M} \leftarrow \{\}$ 
2: for  $i = 1$  to  $N$  do  $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{MS.KeyGen}(1^\lambda)$ 
3:  $(\mu^*, \Sigma^*) \xleftarrow{\$} \mathcal{A}^{\text{Sign}(\text{sk}_1, \text{PK}, \cdot)}(1^\lambda, \text{PK}, \{\text{sk}_j\}_{j=2}^N)$ 
4: if  $\text{MS.MVerify}(\text{PK}, \mu^*, \Sigma^*) = 1$  and  $\mu^* \notin \mathcal{M}$  then
5:   return 1
6: else return 0

```

$\text{Sign}(\text{sk}_1, \text{PK}, \mu)$

```

1: if  $\mu \in \mathcal{M}$  then return  $\perp$ 
2:  $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mu\}$ 
3: return  $\text{MS.Sign}(\text{sk}_1, \text{PK}, \mu)$ 

```

- $b_i \leftarrow \text{MS.Verify}(\text{pk}_i, \text{PK}, \mu, \sigma_i)$: The verification algorithm outputs $b_i = 1$ if σ_i is valid for pk_i and μ .
- $\Sigma \xleftarrow{\$} \text{MS.Agg}(\text{PK}, \mu, \{\sigma_j\}_{j=1}^N)$: The aggregation algorithm creates a multisignature Σ from $\{\sigma_j\}_{j=1}^N$.
- $b \leftarrow \text{MS.MVerify}(\text{PK}, \mu, \Sigma)$: The multisignature verification algorithm outputs $b = 1$ if Σ is valid for PK and μ .

A multisignature scheme is *correct* if all multisignatures created by the algorithms pass the verification, i.e., for all $\lambda, N \in \mathbb{N}$ and $\mu \in \{0, 1\}^*$, it holds, over all possible $(\text{pk}_i, \text{sk}_i)$ generated from $\text{MS.KeyGen}(1^\lambda)$, that $\Pr[\text{MS.MVerify}(\text{PK}, \mu, \Sigma) = 1] = 1$, where Σ is generated by $\text{MS.Agg}(\text{PK}, \mu, \{\text{MS.Sign}(\text{sk}_j, \text{PK}, \mu)\}_{j=1}^N)$.

A multisignature scheme is *existentially unforgeable under one-per-message chosen-message attacks* if for any PPT algorithm $\mathcal{A}$, the advantage $\text{Adv}_{\text{MS}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda) = \Pr[\text{Exp}_{\text{MS}, \mathcal{A}}^{\text{euf-cma}_1}(\lambda)]$ is $\text{negl}(\lambda)$. Generally speaking, the multisignature scheme is secure if the adversary cannot forge a valid multisignature when there is at least one honest signer.

C. Lattices

A lattice is a set of multi-dimensional points with a periodic structure defined as follows.

Definition 1. The lattice generated from ℓ linearly independent vectors $\mathbf{b}_1, \dots, \mathbf{b}_\ell \in \mathbb{R}^\ell$ is defined as $\mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_\ell) = \left\{ \sum_{i=1}^\ell x_i \mathbf{b}_i : x_i \in \mathbb{Z}, 1 \leq i \leq \ell \right\}$, or equivalently $\mathcal{L}(\mathbf{B}) = \{\mathbf{B}\mathbf{x} : \mathbf{x} \in \mathbb{Z}^\ell\}$ where $\mathbf{B} = [\mathbf{b}_1 | \dots | \mathbf{b}_\ell]$.

One particular type of lattices that is of interest is q -ary lattices (i.e., lattices $\mathcal{L}$ where $q\mathbb{Z}^\ell \subseteq \mathcal{L} \subseteq \mathbb{Z}^\ell$) for a prime number q . It is not hard to see that for any $\mathbf{v} \in \mathbb{Z}^\ell$, $\mathbf{v} \in \mathcal{L}$ if and only if $\mathbf{v} \bmod q \in \mathcal{L}$. This property is desirable as it is sufficient to work with q -ary lattices under mod- q arithmetic. In the remainder of this work, all operations involving matrices and vectors are assumed to be under mod- q arithmetic. To construct a q -ary lattice, one can follow Definition 2.

Definition 2. Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ for some prime number q and some $n, m \in \mathbb{Z}$ with $m \geq n$, we define an m -dimensional q -ary lattice $\Lambda^\perp(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{0}\}$.

D. The Short Integer Solution (SIS) Problem

For a vector $\mathbf{v} = [v_1 \dots v_m]^\top \in \mathbb{Z}^m$, the size (ℓ_2 -norm) of $\mathbf{v}$ is denoted by $\|\mathbf{v}\| = \sqrt{v_1^2 + \dots + v_m^2}$. The SIS problem can be defined as follows.

Definition 3 (SIS $_{q,n,m,\beta}$ problem [10]). Given a random matrix $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, find a vector $\mathbf{v} \in \mathbb{Z}^m \setminus \{\mathbf{0}\}$ such that $\mathbf{A}\mathbf{v} = \mathbf{0}$ and $\|\mathbf{v}\| \leq \beta$.

From the analysis in [4], [10], [17], the SIS problem is expected not to be solved in polynomial time when $\beta < \min\left(q, 2^{2\sqrt{n} \log q \log \delta}\right)$ where δ is a constant due to the state-of-the-art lattice reduction algorithms. It is expected that δ is not less than 1.01.

E. Lattice Trapdoors

First, we define the following notations

- $\Lambda_{\mathbf{u}}^\perp(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{u}\}$.
- D_s denotes the discrete Gaussian distribution with a standard deviation s .
- $D_{\Lambda_{\mathbf{u}}^\perp(\mathbf{A}), s}$ denotes D_s restricted to have support $\Lambda_{\mathbf{u}}^\perp(\mathbf{A})$.

In [15], Micciancio and Peikert proposed a construction of trapdoors for lattices and an algorithm for Gaussian preimage sampling for the function $f_{\mathbf{A}}(\mathbf{x}) = \mathbf{A}\mathbf{x}$ as follows.

- 1) $(\mathbf{A}, \mathbf{R}) \xleftarrow{\$} \text{GenTrap}(1^\lambda)$: The trapdoor generation algorithm outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ with its trapdoor $\mathbf{R}$ for a security parameter λ .
- 2) $\mathbf{x} \xleftarrow{\$} \text{SampleD}(\mathbf{R}, \mathbf{A}, \mathbf{u}, s)$: The Gaussian preimage sampling algorithm outputs a vector drawn from a distribution close to $D_{\Lambda_{\mathbf{u}}^\perp(\mathbf{A}), s}$.

The properties of the above algorithms are stated next.

Theorem 4 ([15]). For suitable values of parameters,

- 1) The distribution of $\mathbf{A}$ from $\text{GenTrap}(1^\lambda)$ is $\text{negl}(n)$ -far from uniform.
- 2) For any $\mathbf{u} \in \mathbb{Z}_q^n$, the algorithm $\text{SampleD}(\mathbf{R}, \mathbf{A}, \mathbf{u}, s)$ samples from a distribution within $\text{negl}(n)$ statistical distance of $D_{\Lambda_{\mathbf{u}}^\perp(\mathbf{A}), s}$.

Experiment $\text{Exp}_{\mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{break}}$	$\mathcal{F}(p)$	Experiment $\text{Exp}_{\mathcal{F}, \mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{fork}}$
1: $p \xleftarrow{\$} \mathcal{P}$	1: $\alpha \xleftarrow{\$} \{0, 1\}^c$	1: $p \xleftarrow{\$} \mathcal{P}$
2: $\alpha \xleftarrow{\$} \{0, 1\}^c$	2: $h_1, \dots, h_t \xleftarrow{\$} \mathcal{H}$	2: $(b, s, s') \xleftarrow{\$} \mathcal{F}(p)$
3: $h_1, \dots, h_t \xleftarrow{\$} \mathcal{H}$	3: $(s, k) \xleftarrow{\$} \mathcal{A}(p, h_1, \dots, h_t, \alpha)$	3: return b
4: $(s, k) \xleftarrow{\$} \mathcal{A}(p, h_1, \dots, h_t, \alpha)$	4: $h'_k, \dots, h'_t \xleftarrow{\$} \mathcal{H}$	
5: return k	5: $(s', k') \xleftarrow{\$} \mathcal{A}(p, h_1, \dots, h_{k-1}, h'_k, \dots, h'_t, \alpha)$	
	6: if $k = k'$ and $k \in [t]$ and $h_k \neq h_{k'}$ then	
	7: return $(1, s, s')$	
	8: else return $(0, \perp, \perp)$	

Fig. 1. General forking lemma.

Sig.KeyGen(1^λ)	Sig.Sign(sk, μ)	Sig.Verify(pk, μ, σ)
1: $(\mathbf{A}, \mathbf{R}) \xleftarrow{\$} \text{GenTrap}(1^\lambda)$	1: $\mathbf{r} \leftarrow \text{RO}_1(\text{pk}, \mu)$	1: $\mathbf{r} \leftarrow \text{RO}_1(\text{pk}, \mu)$
2: $\mathbf{S} \xleftarrow{\$} \{-d, \dots, d\}^{m \times k}$	2: $\mathbf{c} \leftarrow \text{RO}_2(\mathbf{r}, \mu)$	2: $\mathbf{c} \leftarrow \text{RO}_2(\mathbf{r}, \mu)$
3: $\mathbf{T} \leftarrow \mathbf{A}\mathbf{S}$	3: $\mathbf{u} \leftarrow \mathbf{T}\mathbf{c} + \mathbf{r}$	3: $\mathbf{u} \leftarrow \mathbf{T}\mathbf{c} + \mathbf{r}$
4: $\text{pk} \leftarrow (\mathbf{A}, \mathbf{T})$	4: $\mathbf{z} \xleftarrow{\$} \text{SampleD}(\mathbf{R}, \mathbf{A}, \mathbf{u}, s)$	4: if $\mathbf{A}\mathbf{z} = \mathbf{u}$ and $\ \mathbf{z}\ \leq 2s\sqrt{m}$ then
5: $\text{sk} \leftarrow (\mathbf{R}, \mathbf{S})$	5: $\sigma \leftarrow \mathbf{z}$	5: return 1
6: return (pk, sk)	6: return σ	6: else return 0

Fig. 2. The proposed signature scheme.

F. General Forking Lemma

The security proofs of signature schemes from the Fiat-Shamir transform are usually based on the general forking lemma. We refer to this lemma as appeared in [18]. Firstly, let $\mathcal{A}$ be an algorithm which receives p sampled from a distribution $\mathcal{P}$, t random elements $h_1, \dots, h_t$ sampled uniformly from $\mathcal{H}$, and a random coin sampled from $\{0, 1\}^c$. The task of $\mathcal{A}$ is to output (s, k) where s is some solution and k is the index of h_k used in the solution, or $k = 0$ in the case that $\mathcal{A}$ cannot construct s from any of $h_1, \dots, h_t$. Based on the definition of $\mathcal{A}$, we define the experiment $\text{Exp}_{\mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{break}}$ as in Fig. 1. We define $\text{break} = \Pr[\text{Exp}_{\mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{break}} \in [t]]$ to be the probability that $\mathcal{A}$ outputs (s, k) and $k \in [t]$.

Next, we consider the forking algorithm $\mathcal{F}$ and the experiment $\text{Exp}_{\mathcal{F}, \mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{fork}}$ in Fig. 1. We note that two executions of $\mathcal{A}$ receive the same value of h_1 up to h_{k-1} . Let $\text{fork} = \Pr[\text{Exp}_{\mathcal{F}, \mathcal{A}, \mathcal{P}, \mathcal{H}, t}^{\text{fork}} = 1]$. The general forking lemma can be stated as follows.

Lemma 5 (General forking lemma [18]). *Let $\mathcal{P}$ be an arbitrary distribution and let $\mathcal{H}$ be the uniform distribution on some finite set H . Let $\mathcal{A}$ be an algorithm (consuming coins from the set $\{0, 1\}^c$). Then, $\text{fork} \geq \frac{1}{t} \cdot \text{break}^2 - \frac{1}{|H|}$.*

III. PROPOSED SIGNATURE SCHEME

A. Description

The proposed signature scheme (Fig. 2) modifies the work of [10] by utilizing lattice trapdoors of Micciancio and Peikert.

The signing and verification algorithms invoke two random oracles $\text{RO}_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$ and $\text{RO}_2 : \{0, 1\}^* \rightarrow \{\mathbf{w} : \mathbf{w} \in \{-1, 0, 1\}^k \wedge \|\mathbf{w}\|_1 \leq \kappa\}$.

The intuition of the proposed scheme is as follows: instead of constructing $\mathbf{r} \leftarrow \mathbf{A}\mathbf{y}$ and $\mathbf{z} \leftarrow \mathbf{S}\mathbf{c} + \mathbf{y}$ from $\mathbf{y}$ which give $\mathbf{A}\mathbf{z} = \mathbf{T}\mathbf{c} + \mathbf{r}$ as done in [10], we invoke the random oracle for $\mathbf{r}$ and then find $\mathbf{z}$ such that $\mathbf{A}\mathbf{z} = \mathbf{T}\mathbf{c} + \mathbf{r}$ without knowing $\mathbf{y}$. We can achieve this since we have the trapdoor $\mathbf{R}$ of $\mathbf{A}$. Two advantages of the proposed scheme are (i) there is no need to restart the protocol because the distribution of our $\mathbf{z}$ is close to $D_{\Lambda_{\mathbf{u}}^\perp(\mathbf{A}), s}$, and (ii) we do not need to include $\mathbf{c}$ in the signature as it can be computed using pk and μ .

B. Correctness Proof

For any signature created by the signing algorithm, we have $\mathbf{A}\mathbf{z} = \mathbf{u}$ as $\mathbf{z}$ is an output of $\text{SampleD}(\mathbf{R}, \mathbf{A}, \mathbf{u}, s)$. And by the following lemma from [10], the probability that $\|\mathbf{z}\| > 2s\sqrt{m}$ is negligible. Therefore, the probability that the valid signature is accepted is close to 1.

Lemma 6 ([10]). $\Pr[\|\mathbf{z}\| > 2s\sqrt{m} \mid \mathbf{z} \xleftarrow{\$} D_s] < 2^{-m}$.

C. Security Proof

We first define the advantage of a PPT algorithm $\mathcal{D}$ in solving $\text{SIS}_{q, n, m, \beta}$ as

$$\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q, n, m, \beta}} = \Pr[\mathbf{v} \neq \mathbf{0} \wedge \mathbf{A}\mathbf{v} = \mathbf{0} \wedge \|\mathbf{v}\| \leq \beta \mid$$

$$\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}; \mathbf{v} \xleftarrow{\$} \mathcal{D}(\mathbf{A})].$$

$\mathcal{B}^{\text{RO}_1, \text{RO}_2}(1^\lambda, \text{pk})$	$\text{ORO}_1(x)$	$\text{ORO}_2(x)$
1: $T_1 \leftarrow []$	1: parse x as (pk, μ)	1: if $T_2[x] = \perp$ then
2: $T_2 \leftarrow []$	2: if $T_1[(\text{pk}, \mu)] = \perp$ then	2: $T_2[x] \leftarrow \text{RO}_2(x)$
3: $(\mu^*, \sigma^*) \xleftarrow{\$} \mathcal{C}^{\text{ORO}_1, \text{ORO}_2, \text{OSign}}(1^\lambda, \text{pk})$	3: $\mathbf{c} \xleftarrow{\$} H; \mathbf{z} \xleftarrow{\$} D_s$	3: return $T_2[x]$
4: return (μ^*, σ^*)	4: $T_1[(\text{pk}, \mu)] \leftarrow (\mathbf{c}, \mathbf{z})$	
	5: $(\mathbf{c}, \mathbf{z}) \leftarrow T_1[(\text{pk}, \mu)]$	$\text{OSign}(\mu)$
	6: $\mathbf{r} \leftarrow \mathbf{A}\mathbf{z} - \mathbf{T}\mathbf{c}$	1: $\mathbf{r} \leftarrow \text{ORO}_1(\text{pk}, \mu)$
	7: if $T_2[(\mathbf{r}, \mu)] \notin \{\perp, \mathbf{c}\}$ then abort	2: $(\mathbf{c}, \mathbf{z}) \leftarrow T_1[(\text{pk}, \mu)]$
	8: $T_2[(\mathbf{r}, \mu)] \leftarrow \mathbf{c}$	3: return $\mathbf{z}$
	9: return $\mathbf{r}$	

Fig. 3. The algorithms simulating signature queries.

The proof consists of two lemmas, following the method used in [18]. The first lemma shows the advantage of a key-only attacker $\mathcal{B}$ in terms of that of $\mathcal{D}$, and the second lemma shows that a key-only attacker $\mathcal{B}$ can be reduced to a one-message chosen-message attacker $\mathcal{C}$. The advantage of a key-only attacker is defined similar to that of a chosen-message attacker except that the key-only attacker cannot make a signing query:

$$\begin{aligned} \text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{euf-ko}}(\lambda) &= \Pr[\text{Sig.Verify}(\text{pk}, \mu^*, \sigma^*) = 1 \mid \\ &\quad (\text{pk}, \text{sk}) \xleftarrow{\$} \text{Sig.KeyGen}(1^\lambda); \\ &\quad (\mu^*, \sigma^*) \xleftarrow{\$} \mathcal{B}(1^\lambda, \text{pk})]. \end{aligned}$$

In all subsequent proofs, we denote the range of RO_2 by $H = \{\mathbf{w} : \mathbf{w} \in \{-1, 0, 1\}^k \wedge \|\mathbf{w}\|_1 \leq \kappa\}$.

Lemma 7. *For any PPT adversary $\mathcal{B}$ against the proposed signature scheme, making at most t_2 random oracle queries to RO_2 , there is a PPT algorithm $\mathcal{D}$ solving $\text{SIS}_{q,n,m,\beta}$ for $\beta = (4s + 2d\kappa)\sqrt{m}$ such that*

$$\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q,n,m,\beta}} \geq \frac{1}{2} \left(\frac{1}{t_2 + 1} \cdot \left(\text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{euf-ko}}(\lambda) \right)^2 - \frac{1}{|H|} \right).$$

Proof. Let $\mathbf{A}$ be a given instance of $\text{SIS}_{q,n,m,\beta}$. We choose $\mathbf{S} \xleftarrow{\$} \{-d, \dots, d\}^{m \times k}$, compute $\mathbf{T} \leftarrow \mathbf{A}\mathbf{S}$, and send $\text{pk} = (\mathbf{A}, \mathbf{T})$ to $\mathcal{B}$. Since the distribution of $\mathbf{A}$ from $\text{GenTrap}(1^\lambda)$ is statistically close to uniform, $\mathcal{B}$ cannot distinguish between two cases and see pk as a valid public key.

Next, we use the result of the general forking lemma where $\mathcal{P}$ is the set of all public keys. For an output (μ^*, σ^*) of $\mathcal{B}$, we can assume that $\text{RO}_2(\text{RO}_1(\text{pk}, \mu^*), \mu^*)$ is queried during its execution, otherwise we can make this extra query, resulting in at most $t_2 + 1$ random oracle queries to RO_2 . We then apply Lemma 5 with $\text{break} = \text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{euf-ko}}(\lambda)$ and $t = t_2 + 1$.

Suppose the outputs of two executions of $\mathcal{B}$ in the forking algorithm $\mathcal{F}$ are $(\mu_1^*, \sigma_1^* = \mathbf{z})$ and $(\mu_2^*, \sigma_2^* = \mathbf{z}')$. By the general forking lemma, two executions are the same up to, but not include, $\mathbf{c} = \text{RO}_2(\text{RO}_1(\text{pk}, \mu_1^*), \mu_1^*)$ and $\mathbf{c}' = \text{RO}_2(\text{RO}_1(\text{pk}, \mu_2^*), \mu_2^*)$ with $\mathbf{c} \neq \mathbf{c}'$. This implies that

$\text{RO}_1(\text{pk}, \mu_1^*) = \text{RO}_1(\text{pk}, \mu_2^*) = \mathbf{r}$. Therefore, $\mathbf{A}\mathbf{z} - \mathbf{T}\mathbf{c} = \mathbf{A}\mathbf{z}' - \mathbf{T}\mathbf{c}'$. From $\mathbf{T} = \mathbf{A}\mathbf{S}$, we have $\mathbf{A}(\mathbf{z} - \mathbf{z}' - \mathbf{S}(\mathbf{c} - \mathbf{c}')) = \mathbf{0}$.

By the parameter choices, $\|\mathbf{z} - \mathbf{z}' - \mathbf{S}(\mathbf{c} - \mathbf{c}')\| \leq \beta$. In the case that $\mathbf{z} - \mathbf{z}' - \mathbf{S}(\mathbf{c} - \mathbf{c}') = \mathbf{0}$, we use the fact shown in [10] that, with very high probability, there exists $\mathbf{S}'$ such that $\mathbf{T} = \mathbf{A}\mathbf{S} = \mathbf{A}\mathbf{S}'$ and $\mathbf{z} - \mathbf{z}' - \mathbf{S}'(\mathbf{c} - \mathbf{c}') \neq \mathbf{0}$. Because $\mathcal{B}$ does not have any knowledge about sk , there is a probability of at least $\frac{1}{2}$ such that $\text{sk} = \mathbf{S}'$ and not $\mathbf{S}$ when we choose it, which enables us to construct a non-zero vector that is a solution to $\text{SIS}_{q,n,m,\beta}$. Thus, $\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q,n,m,\beta}} \geq \frac{1}{2} \cdot \text{fork}$. $\square$

Lemma 8. *For any PPT adversary $\mathcal{C}$ against the proposed signature scheme, making at most t_1 (resp. t_2) random oracle queries to RO_1 (resp. RO_2) and t_s distinct signature queries, there exists a PPT adversary $\mathcal{B}$ against the scheme, making at most t_2 random oracle queries to RO_2 , such that*

$$\text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{euf-ko}}(\lambda) \geq \text{Adv}_{\text{Sig}, \mathcal{C}}^{\text{euf-cma}_1}(\lambda) - \frac{(t_1 + t_s)(t_1 + t_2 + t_s)}{q^n}.$$

Proof. Given $\mathcal{C}$, we can construct $\mathcal{B}$ by “simulating” signature queries as shown in Fig. 3. Our goal is, without knowing the trapdoor $\mathbf{R}$ of $\mathbf{A}$, programming the random oracles so that

$$\mathbf{A} \cdot \underbrace{\text{OSign}(\mu)}_{\mathbf{z}} = \mathbf{T} \cdot \underbrace{\text{ORO}_2(\overbrace{\text{ORO}_1(\text{pk}, \mu)}^{\mathbf{r}}, \mu)}_{\mathbf{c}} + \overbrace{\text{ORO}_1(\text{pk}, \mu)}^{\mathbf{r}}.$$

We can see that for a fixed value of $\mathbf{r}$, the value of $\mathbf{c}$ depends on $\mathbf{r}$, and the value of $\mathbf{z}$ then depends on both $\mathbf{r}$ and $\mathbf{c}$. Thus, when $\mathcal{C}$ invokes ORO_1 for $\mathbf{r}$, we need to compute a tuple $(\mathbf{r}, \mathbf{c}, \mathbf{z})$ that satisfies $\mathbf{A}\mathbf{z} = \mathbf{T}\mathbf{c} + \mathbf{r}$ and program the random oracles accordingly. The algorithm ORO_1 first samples $(\mathbf{c}, \mathbf{z})$ (if there is no existing value), computes $\mathbf{r} \leftarrow \mathbf{A}\mathbf{z} - \mathbf{T}\mathbf{c}$, programs $\text{ORO}_2(\mathbf{r}, \mu) = \mathbf{c}$, and then returns $\mathbf{r}$. The value $(\mathbf{c}, \mathbf{z})$ is stored to answer the signing query. The algorithm ORO_2 returns the value that is programmed by ORO_1 or a result from RO_2 if no value is set, and the algorithm OSign invokes ORO_1 to create a tuple $(\mathbf{r}, \mathbf{c}, \mathbf{z})$ if necessary and then returns $\mathbf{z}$. The simulation works correctly since the equation $\mathbf{A}\mathbf{z} = \mathbf{T}\mathbf{c} + \mathbf{r}$ is satisfied and the distribution of $(\mathbf{r}, \mathbf{c}, \mathbf{z})$ is the same as that obtaining from the genuine signer.

$\text{SV.Init}(1^\lambda)$ 1 : $(\mathbf{A}, \mathbf{R}) \xleftarrow{\$} \text{GenTrap}(1^\lambda)$ 2 : $\text{pp} \leftarrow \mathbf{A}$ 3 : return pp		$\text{SV.Sample}(\text{sk}_i, \text{pk}_i, \mu)$ 1 : if $\mathbf{T}_i \neq \mathbf{A}\mathbf{S}_i$ then return $\perp$ 2 : $\mathbf{r}_i \leftarrow \text{RO}_1(\text{pk}_i, \mu)$ 3 : return $\text{SampleD}(\mathbf{R}, \mathbf{A}, \mathbf{r}_i, s)$	
$\text{MS.KeyGen}(\text{pp})$ 1 : $\mathbf{S}_i \xleftarrow{\$} \{-d, \dots, d\}^{m \times k}$ 2 : $\mathbf{T}_i \leftarrow \mathbf{A}\mathbf{S}_i$ 3 : $\text{sk}_i \leftarrow \mathbf{S}_i$ 4 : $\text{pk}_i \leftarrow \mathbf{T}_i$ 5 : return $(\text{sk}_i, \text{pk}_i)$	$\text{MS.Sign}(\text{sk}_i, \text{PK}, \mu)$ 1 : $\mathbf{r} \leftarrow \sum_{j=1}^N \text{RO}_1(\text{pk}_j, \mu)$ 2 : $\mathbf{c}_i \leftarrow \text{RO}_2(\mathbf{r}, \text{pk}_i, \mu)$ 3 : $\mathbf{y}_i \xleftarrow{\$} \text{SV.Sample}(\text{sk}_i, \text{pk}_i, \mu)$ 4 : $\mathbf{z}_i \leftarrow \mathbf{S}_i \mathbf{c}_i + \mathbf{y}_i$ 5 : $\sigma_i \leftarrow \mathbf{z}_i$ 6 : Go to 3 with probability $\text{Rej}(\mathbf{S}_i \mathbf{c}_i, \mathbf{z}_i)$ 7 : return σ_i	$\text{MS.Verify}(\text{pp}, \text{pk}_i, \text{PK}, \mu, \sigma_i)$ 1 : $\mathbf{r}_i \leftarrow \text{RO}_1(\text{pk}_i, \mu)$ 2 : $\mathbf{r} \leftarrow \sum_{j=1}^N \text{RO}_1(\text{pk}_j, \mu)$ 3 : $\mathbf{c}_i \leftarrow \text{RO}_2(\mathbf{r}, \text{pk}_i, \mu)$ 4 : $\mathbf{u}_i \leftarrow \mathbf{T}_i \mathbf{c}_i + \mathbf{r}_i$ 5 : if $\mathbf{A}\mathbf{z}_i = \mathbf{u}_i$ and $\ \mathbf{z}_i\ \leq 2s\sqrt{m}$ then 6 : return 1 7 : else return 0	
$\text{MS.Agg}(\text{pp}, \text{PK}, \mu, \{\sigma_j\}_{j=1}^N)$ 1 : for $i = 1$ to N do 2 : if $\text{MS.Verify}(\text{pp}, \text{pk}_i, \text{PK}, \mu, \sigma_i) = 0$ then return $\perp$ 3 : $\mathbf{z} \leftarrow \sum_{j=1}^N \mathbf{z}_j$ 4 : $\Sigma \leftarrow \mathbf{z}$ 5 : return Σ		$\text{MS.MVerify}(\text{pp}, \text{PK}, \mu, \Sigma)$ 1 : $\mathbf{r} \leftarrow \sum_{j=1}^N \text{RO}_1(\text{pk}_j, \mu)$ 2 : for $i = 1$ to N do $\mathbf{c}_i \leftarrow \text{RO}_2(\mathbf{r}, \text{pk}_i, \mu)$ 3 : $\mathbf{u} \leftarrow \sum_{j=1}^N \mathbf{T}_j \mathbf{c}_j + \mathbf{r}$ 4 : if $\mathbf{A}\mathbf{z} = \mathbf{u}$ and $\ \mathbf{z}\ \leq 2Ns\sqrt{m}$ then return 1 5 : else return 0	

Fig. 4. The proposed multisignature scheme.

The only case that $\mathcal{B}$ fails is when $\mathcal{B}$ aborts in Line 7 of ORO_1 . This happens when $T_2[(\mathbf{r}, \mu)]$ is set to a different value by any of the algorithms. We upper-bound the probability that this happens by the probability that $T_2[(\mathbf{r}, \cdot)]$ is set for any message. Since the distribution of $\mathbf{r}$ is close to uniform in $\mathbb{Z}_q^n$, the probability that we encounter the same $\mathbf{r}$ is $\frac{1}{q^n}$. By the union bound, the probability that $T_2[(\mathbf{r}, \cdot)]$ is set in the previous algorithm invocations is at most $(t_1 + t_2 + t_s) \cdot \frac{1}{q^n}$. Because $\mathcal{B}$ can fail when ORO_1 or OSign is called, the probability that $\mathcal{B}$ fails is at most $(t_1 + t_s)(t_1 + t_2 + t_s) \cdot \frac{1}{q^n}$. $\square$

We conclude the security proof as the following theorem.

Theorem 9. *For any PPT adversary $\mathcal{C}$ against the proposed signature scheme, making at most t_1 (resp. t_2) random oracle queries to RO_1 (resp. RO_2) and t_s distinct signature queries, there is a PPT algorithm $\mathcal{D}$ solving $\text{SIS}_{q,n,m,\beta}$ for $\beta = (4s + 2d\kappa)\sqrt{m}$ such that*

$$\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q,n,m,\beta}} \geq \frac{1}{2} \left(\frac{1}{t_2 + 1} \left(\text{Adv}_{\text{Sig}, \mathcal{C}}^{\text{euf-cma}_1}(\lambda) - \epsilon \right)^2 - \frac{1}{|H|} \right)$$

where $\epsilon = (t_1 + t_s)(t_1 + t_2 + t_s)/q^n$.

IV. PROPOSED MULTISIGNATURE SCHEME

A. Description

In this section, we extend our proposed signature scheme in the previous section to the multisignature scheme shown in

Fig. 4. The variable $1 \leq i \leq N$ denotes the index of a signer.

In the scheme, all signers use the same $\mathbf{A}$, similar to previous works. However, we cannot give the trapdoor $\mathbf{R}$ to each signer as any signer can forge a valid signature of other signers and thus a valid multisignature. Instead, $\mathbf{R}$ is kept with the honest centralized server (called SV) and each signer then communicates with the server when SampleD is needed.

Nevertheless, the scheme is not secure if a signer can obtain the preimage of any value. For example, in MS.MVerify , the value of $\mathbf{u}$ can be publicly computed and the scheme is broken if any signer can ask the server for the preimage of $\mathbf{u}$, which is indeed a valid multisignature. To solve this issue, we restrict that each signer can only obtain the preimage of the value of the form $\text{RO}_1(\text{pk}_i, \mu)$. This is done by the signer submitting pk_i and μ and the server confirming that the signer has the corresponding sk_i , e.g., using an identification scheme.

Furthermore, because we now Gaussian sample the preimage of $\mathbf{r}_i$ instead of $\mathbf{z}_i$ as done in the signature scheme, $\mathbf{z}_i = \mathbf{S}_i \mathbf{c}_i + \mathbf{y}_i$ might leak an information about $\mathbf{S}_i \mathbf{c}_i$. Referring to [10], we need to resample $\mathbf{y}_i$ before outputting $\mathbf{z}_i$ with probability $\text{Rej}(\mathbf{S}_i \mathbf{c}_i, \mathbf{z}_i) = 1 - \min \left(\frac{D_s(\mathbf{z}_i)}{M \cdot D_s(\mathbf{S}_i \mathbf{c}_i)(\mathbf{z}_i)}, 1 \right)$ where $D_s(\mathbf{S}_i \mathbf{c}_i)$ denotes D_s centered at $\mathbf{S}_i \mathbf{c}_i$ instead of $\mathbf{0}$, and M is a suitable constant. By this restart, it is proved that the distribution of $\mathbf{z}_i$ is close to D_s . Apart from what mentioned, everything is the same as the signature scheme: each signer

$\text{XSample}(\text{sk}_i, \text{pk}_i, \mu)$	$\text{XRO}_1(x)$	
1: if $\mathbf{T}_i \neq \mathbf{AS}_i$ then return $\perp$	1: parse x as (pk_i, μ)	
2: if $T_1[(\text{pk}_i, \mu)] = \perp$ then	2: if $T_1[(\text{pk}_i, \mu)] = \perp$ then	
3: $\mathbf{r}_i \leftarrow \text{XRO}_1(\text{pk}_i, \mu)$	3: $T_1[(\text{pk}_i, \mu)] \xleftarrow{\$} D_s$	
4: return $T_1[(\text{pk}_i, \mu)]$	4: return $\mathbf{A} \cdot T_1[(\text{pk}_i, \mu)]$	

$\mathcal{B}^{\text{XRO}_1, \text{RO}_2}(1^\lambda, \text{PK}, \{\text{sk}_j\}_{j=2}^N)$	$\text{ORO}_1(x)$	$\text{ORO}_2(x)$
1: $T'_1 \leftarrow []$	1: parse x as (pk, μ)	1: if $T_2[x] = \perp$ then
2: $T_2 \leftarrow []$	2: if $\text{pk} \neq \text{pk}_1$ then return $\text{XRO}_1(x)$	2: $T_2[x] \leftarrow \text{RO}_2(x)$
3: $(\mu^*, \Sigma^*) \xleftarrow{\$}$ $\mathcal{C}^{\text{ORO}_1, \text{ORO}_2, \text{OSign}}(1^\lambda, \text{PK}, \{\text{sk}_j\}_{j=2}^N)$	3: if $T'_1[(\text{pk}_1, \mu)] = \perp$ then	3: return $T_2[x]$
4: return (μ^*, Σ^*)	4: $\mathbf{c}_1 \xleftarrow{\$} H; \mathbf{z}_1 \xleftarrow{\$} D_s$	
	5: $T'_1[(\text{pk}_1, \mu)] \leftarrow (\mathbf{c}_1, \mathbf{z}_1)$	$\text{OSign}(\mu)$
	6: $(\mathbf{c}_1, \mathbf{z}_1) \leftarrow T'_1[(\text{pk}_1, \mu)]$	1: $\mathbf{r}_1 \leftarrow \text{ORO}_1(\text{pk}_1, \mu)$
	7: $\mathbf{r}_1 \leftarrow \mathbf{A}\mathbf{z}_1 - \mathbf{T}_1\mathbf{c}_1$	2: $(\mathbf{c}_1, \mathbf{z}_1) \leftarrow T'_1[(\text{pk}_1, \mu)]$
	8: $\mathbf{r} \leftarrow \mathbf{r}_1 + \sum_{j=2}^N \text{ORO}_1(\text{pk}_j, \mu)$	3: return $\mathbf{z}_1$
	9: if $T_2[(\mathbf{r}, \text{pk}_1, \mu)] \notin \{\perp, \mathbf{c}_1\}$ then abort	
	10: $T_2[(\mathbf{r}, \text{pk}_1, \mu)] \leftarrow \mathbf{c}_1$	
	11: return $\mathbf{r}_1$	

Fig. 5. The algorithms simulating multisignature queries.

constructs $\mathbf{z}_i$ where $\mathbf{A}\mathbf{z}_i = \mathbf{T}_i\mathbf{c}_i + \mathbf{r}_i$ and the multisignature is $\Sigma = \mathbf{z} = \sum_{j=1}^N \mathbf{z}_j$ where $\mathbf{A}\mathbf{z} = \sum_{j=1}^N \mathbf{T}_j\mathbf{c}_j + \mathbf{r}_j$.

We point out that, unlike previous works, there is no broadcasting among signers and they do not need to be online at the same time. For MS.Agg, anyone, signer or not, can execute it and each signer only needs to send σ_i to this person, thus no broadcasting is required. This makes the signing process more flexible. Moreover, if any signer needs to resample $\mathbf{y}_i$ because of the rejection, other signers do not have to resample theirs. This is different from all previous works where all signers have to restart the protocol when any signer needs to, the process which implies additional cost. Thus, our multisignature scheme reduces the communication cost in two aspects. The only requirements of the proposed scheme are the honest server and a secure communication channel between the server and each signer.

B. Correctness Proof

From the definition of MS.Sign, $\mathbf{A}\mathbf{z}_i = \mathbf{u}_i$ must hold for all $1 \leq i \leq N$. Considering Lines 1–2 of MS.Agg, the aggregation algorithm returns $\perp$ if any signature does not pass the verification. In the case that $\Sigma \neq \perp$, we know that $\mathbf{A}\mathbf{z} = \mathbf{A} \sum_{j=1}^N \mathbf{z}_j = \sum_{j=1}^N \mathbf{u}_j = \mathbf{u}$ and $\|\mathbf{z}_i\| \leq 2s\sqrt{m}$. Thus, $\|\mathbf{z}\| \leq 2Ns\sqrt{m}$ and Σ is a valid multisignature.

The only case that MS.Agg returns $\perp$ is when there exists $1 \leq i \leq N$ such that $\|\mathbf{z}_i\| > 2s\sqrt{m}$. By Lemma 6 and the union bound, this event happens with probability at most $N2^{-m}$, which is negligible. Therefore, the multisignature from MS.Agg is accepted with probability close to 1.

C. Security Proof

The security proof for the proposed multisignature scheme is more complicated comparing to the signature scheme. This is because, given an instance $\mathbf{A}$ of $\text{SIS}_{q,n,m,\beta}$, we need to provide the results of SV.Sample for key pairs owned by the adversary without having its trapdoor $\mathbf{R}$. We again deal with this using a simulation as shown in Fig. 5.

Lemma 10. *For any PPT adversary $\mathcal{B}$ against the proposed multisignature scheme, making at most t_2 random oracle queries to RO_2 , there is a PPT algorithm $\mathcal{D}$ solving $\text{SIS}_{q,n,m,\beta}$ for $\beta = (4Ns + 2d\kappa)\sqrt{m}$ such that*

$$\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q,n,m,\beta}} \geq \frac{1}{2} \left(\frac{1}{t_2 + 1} \cdot \left(\text{Adv}_{\text{MS}, \mathcal{B}}^{\text{euf-ko}}(\lambda) \right)^2 - \frac{1}{|H|} \right).$$

Proof. For an instance $\mathbf{A}$ of $\text{SIS}_{q,n,m,\beta}$ and $1 \leq i \leq N$, we choose $\mathbf{S}_i \xleftarrow{\$} \{-d, \dots, d\}^{m \times k}$, compute $\mathbf{T}_i \leftarrow \mathbf{AS}_i$, and send PK and $\{\text{sk}_j\}_{j=2}^N$ to $\mathcal{B}$. We only give $\mathcal{B}$ an access to XSample and XRO_1 shown in Fig. 5. Distinguishing between RO_1 and XRO_1 is similar to the problem of [10, Definition 2.3] which is computationally hard. $\mathcal{B}$ can normally invoke RO_2 .

This time we are interested in two runs of the adversary where the value of $\mathbf{c}_1 = \text{RO}_2(\mathbf{r}, \text{pk}_1, \mu)$ are different and $\mathbf{c}_i$ for $2 \leq i \leq N$ are the same. Let the two outputs of the forking algorithm be $(\mu_1^*, \Sigma_1^* = \mathbf{z})$ and $(\mu_2^*, \Sigma_2^* = \mathbf{z}')$ and the outputs of RO_2 be $\mathbf{c}_i$ and $\mathbf{c}'_i$. Since two runs are the same up to, but not include, the computation of $\mathbf{c}_1$ and $\mathbf{c}'_1$, the value of $\mathbf{r}_i$ are the same for all $1 \leq i \leq N$. Thus, $\mathbf{r} = \mathbf{A}\mathbf{z} - \sum_{j=1}^N \mathbf{T}_j\mathbf{c}_j =$

$\mathbf{A}\mathbf{z}' - \sum_{j=1}^N \mathbf{T}_j \mathbf{c}'_j$. Since $\mathbf{c}_i = \mathbf{c}'_i$ for $2 \leq i \leq N$, we have $\mathbf{A}(\mathbf{z} - \mathbf{z}' - \mathbf{S}_1(\mathbf{c}_1 - \mathbf{c}'_1)) = \mathbf{0}$. The same reasoning as in Lemma 7 is then applied to achieve the desired bound. $\square$

Lemma 11. *For any PPT adversary $\mathcal{C}$ against the proposed multisignature scheme, making at most t_1 (resp. t_2) random oracle queries to RO_1 (resp. RO_2) and t_s distinct signature queries, there exists a PPT adversary $\mathcal{B}$ against the scheme, making at most t_2 random oracle queries to RO_2 , such that*

$$\text{Adv}_{\text{MS}, \mathcal{B}}^{\text{euf-ko}}(\lambda) \geq \text{Adv}_{\text{MS}, \mathcal{C}}^{\text{euf-cma}_1}(\lambda) - \frac{(t_1 + t_s)(t_1 + t_2 + t_s)}{q^n}.$$

Proof. The proof follows the same approach as in Lemma 8 except that $\mathcal{C}$ has an access to XSample , ORO_1 (which calls XRO_1), ORO_2 , and OSign . $\square$

We finally obtain the theorem describing the security of our multisignature scheme as follows.

Theorem 12. *For any PPT adversary $\mathcal{C}$ against the proposed multisignature scheme, making at most t_1 (resp. t_2) random oracle queries to RO_1 (resp. RO_2) and t_s distinct signature queries, there is a PPT algorithm $\mathcal{D}$ solving $\text{SIS}_{q,n,m,\beta}$ for $\beta = (4Ns + 2d\kappa)\sqrt{m}$ such that*

$$\text{Adv}_{\mathcal{D}}^{\text{SIS}_{q,n,m,\beta}} \geq \frac{1}{2} \left(\frac{1}{t_2 + 1} \left(\text{Adv}_{\text{MS}, \mathcal{C}}^{\text{euf-cma}_1}(\lambda) - \epsilon \right)^2 - \frac{1}{|H|} \right)$$

where $\epsilon = (t_1 + t_s)(t_1 + t_2 + t_s)/q^n$.

Remark. *The proposed multisignature scheme can be modified to produce an aggregated public key, which is a single public key representing the whole group of signers. We refer the interested readers to [3], [6] for more details.*

V. CONCLUSION

We have presented a lattice-based multisignature scheme that eliminates the need of broadcasting information among signers. This improvement is due to lattice trapdoors. By the result of this work, it is interesting to utilize lattice trapdoors to improve other lattice-based cryptosystems in a similar manner.

Even though the communication cost is reduced, we need an honest server to keep signers from knowing the trapdoor and forging a multisignature. It is not trivial to remove the server in our scheme by lattice primitives that we know and we therefore leave this as an open question.

Our future work is to consider other security models that are stronger than the existentially unforgeable under one-per-message chosen-message attack such as the knowledge of the secret key model, the certified key model, the key verification model, and the plain public-key model. Also, we plan to instantiate all the parameters of the schemes so that we can compare other aspects of our schemes with previous works based on the same security level.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their constructive comments regarding the security of the proposed schemes. The first author would like to thank Nina Bindel for her advice on the proofs, and Kittiphop Phalakarn for his suggestions on the schemes. The first author is supported by the Ripple Impact Fund through a Ripple Graduate Fellowship. The second author is supported by JSPS Grant-in-Aid for Transformative Research Areas A grant number JP21H05845.

REFERENCES

- [1] K. Itakura and K. Nakamura, "A public-key cryptosystem suitable for digital multisignatures," *NEC Research & Development*, no. 71, pp. 1–8, 1983.
- [2] M. Bellare and G. Neven, "Identity-based multi-signatures from RSA," in *Cryptographers' Track at the RSA Conference*, pp. 145–162, Springer, 2007.
- [3] D. Boneh, M. Drijvers, and G. Neven, "Compact multi-signatures for smaller blockchains," in *International Conference on the Theory and Application of Cryptology and Information Security*, pp. 435–464, Springer, 2018.
- [4] D. Micciancio and O. Regev, "Lattice-based cryptography," in *Post-quantum cryptography*, pp. 147–191, Springer, 2009.
- [5] R. El Bansarkhani and J. Sturm, "An efficient lattice-based multisignature scheme with applications to bitcoins," in *International Conference on Cryptology and Network Security*, pp. 140–155, Springer, 2016.
- [6] C. Ma and M. Jiang, "Practical lattice-based multisignature schemes for blockchains," *IEEE Access*, vol. 7, pp. 179765–179778, 2019.
- [7] M. Fukumitsu and S. Hasegawa, "A lattice-based provably secure multisignature scheme in quantum random oracle model," in *International Conference on Provable Security*, pp. 45–64, Springer, 2020.
- [8] I. Damgård, C. Orlandi, A. Takahashi, and M. Tibouchi, "Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices," in *IACR International Conference on Public-Key Cryptography*, pp. 99–130, Springer, 2021.
- [9] V. Lyubashevsky, "Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures," in *International Conference on the Theory and Application of Cryptology and Information Security*, pp. 598–616, Springer, 2009.
- [10] V. Lyubashevsky, "Lattice signatures without trapdoors," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 738–755, Springer, 2012.
- [11] T. Güneysu, V. Lyubashevsky, and T. Pöppelmann, "Practical lattice-based cryptography: A signature scheme for embedded systems," in *International Workshop on Cryptographic Hardware and Embedded Systems*, pp. 530–547, Springer, 2012.
- [12] Z. Jaroucheh, B. Ghaleb, and W. J. Buchanan, "SkIcCoin: Toward a scalable proof-of-stake and collective signature based consensus protocol for strong consistency in blockchain," in *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*, pp. 143–150, IEEE, 2020.
- [13] M. Kansal and R. Dutta, "Round optimal secure multisignature schemes from lattice with public key aggregation and signature compression," in *International Conference on Cryptology in Africa*, pp. 281–300, Springer, 2020.
- [14] Z.-Y. Liu, Y.-F. Tseng, and R. Tso, "Cryptanalysis of a round optimal lattice-based multisignature scheme," *IACR Cryptol. ePrint Arch.*, 2020.
- [15] D. Micciancio and C. Peikert, "Trapdoors for lattices: Simpler, tighter, faster, smaller," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 700–718, Springer, 2012.
- [16] E. Kiltz, V. Lyubashevsky, and C. Schaffner, "A concrete treatment of fiat-shamir signatures in the quantum random-oracle model," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 552–586, Springer, 2018.
- [17] N. Gama and P. Q. Nguyen, "Predicting lattice reduction," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 31–51, Springer, 2008.
- [18] A. Mittelbach and M. Fischlin, *The Theory of Hash Functions and Random Oracles: An Approach to Modern Cryptography*. Springer Nature, 2021.